EXP 6

CODE:

```
import heapq

def a_star(graph, start, goal, heuristic):
    # Priority queue: (f_score, g_score, node, path)
    pq = [(heuristic[start], 0, start, [start])]
    visited = set()

    while pq:
        f, g, node, path = heapq.heappop(pq)
        if node == goal:
            return path, g  # path and total cost
        if node in visited:
            continue
        visited.add(node)

        for neighbor, cost in graph[node]:
            if neighbor not in visited:
                g_new = g + cost
                f_new = g_new + heuristic[neighbor]
                heapq.heappush(pq, (f_new, g_new, neighbor, path + [neighbor]))
    return None, float("inf")

# Example graph (adjacency list with costs)
graph = {
    'A': [('B', 1), ('C', 4)],
    'B': [('C', 2), ('D', 5)],
```

```
    'C': [('D', 1)],
    'D': []
}

# Example heuristic values (straight-line estimates to goal 'D')
heuristic = {
    'A': 7,
    'B': 6,
    'C': 2,
    'D': 0
}

path, cost = a_star(graph, 'A', 'D', heuristic)
print("Optimal Path:", path)
print("Total Cost:", cost)
```

OUTPUT:

